

Modular Design for UKG's 12-Step Workflow and 17-Axis System

12-Step Refinement Workflow – Each Step as a Module vs Monolith

It is **highly recommended to implement each of the 12 refinement steps as separate modules** (or components) rather than one monolithic block. Each step in the refinement QA loop has a distinct purpose (AoT reasoning chain, ToT branching, gap analysis, validation, etc.), so isolating them into individual Python modules or functions makes the system more maintainable and extensible ¹. This modular approach aligns with UKG's design philosophy: in the core reasoning pipeline, each stage is treated as a distinct function or workflow step, coordinated by a central engine ¹. The same should apply to the refinement steps.

Benefits of per-step modules:

- **Separation of Concerns:** Each step's logic (e.g. logic auditing vs. bias checking) lives in its own module, making it easier to develop and test in isolation. For example, you might have `step01_aot.py` for **Algorithm of Thought** and `step04_data_validation.py` for **Data Validation**, each implementing a common interface (e.g. a `Step.run(context)` method). This was reflected in the proposed project structure where each refinement step was a standalone component ¹.
- **Plug-and-Play Extensibility:** New refinement checks or improvements can be added as new modules without disrupting others, as long as they adhere to the context input/output contract. This modular design means developers can “plug in” improvements to each step independently ¹. For instance, adding a new Step 13 for stylistic enhancements would just mean writing a new module and adding it to the sequence.
- **Reuse and Testing:** If some steps (like Data Validation or Ethics Check) could be reused in other workflows, having them as separate modules or even microservices with an API makes that possible. Each module can be unit-tested with a mock context to ensure it performs its function (e.g., flagging inconsistent facts) correctly.

Integration into the full system: All these step modules are tied together by a **Refinement Orchestrator** that runs the steps sequentially as a sub-workflow. In practice, you might implement an orchestrator class that holds an ordered list of Step objects and iterates through them, updating a shared `RefinementContext` object. After the main answer is generated by the 10-layer simulation stack, the orchestrator feeds the draft answer through Step 1, 2, ... 12 in order ². At each step, the module can append its findings (e.g. any corrections or improvements) to the context. By Step 11, the answer is rewritten and scored, and Step 12 performs the memory patch if confidence is high enough. The orchestrator can then check the final confidence score and decide if another refinement iteration is needed (or if it should expand context and retry) ². This loop runs until the answer confidence meets the threshold (e.g. ≥ 0.995) or a max iteration count is reached, as described in the design.

If you need to **expose the refinement workflow via APIs**, you have a couple of options:

- **Single Endpoint for the Orchestrator:** The simplest approach is to expose one API endpoint (or method) that triggers the entire 12-step refinement process. Under the hood, it will call each step's module in sequence. For example, a web service could accept a draft answer and return the refined answer after running all steps.
- **Microservice per Step:** Alternatively, each step could be a microservice with its own endpoint (e.g. `/refine/aot`, `/refine/tot`, ... `/refine/memory_patch`). The orchestrator would then call these endpoints in order. This adds overhead but can be useful if steps need to scale independently or use different resources. The UKG architecture does allow for such a microservice-style runtime – the Execution layer's orchestrator can route subtasks to different internal workers or external services as needed ². In practice, unless you have a strong scaling or separation requirement, keeping the 12 steps as in-process modules is simpler.

Building it into the system: In code, you might define a `RefinementContext` dataclass to carry the question, draft answer, current answer state, evidence, and confidence score through the steps. Each step module would import this context and implement a `run(ctx: RefinementContext) -> RefinementContext` function. You would register all step modules in order, then in the orchestrator do something like:

```
ctx = RefinementContext(query, draft_answer)
for step in refinement_steps:
    ctx = step.run(ctx)
# check ctx.confidence, potentially loop again or exit
```

This approach was outlined in the design discussion – the system defines a **refinement_engine** configuration listing the 12 steps, and the master workflow invokes it after the main reasoning cycle ³. By treating the refinement as a contained sub-workflow, the main orchestrator (the 10-layer simulation engine) can call it when needed (for example, after composing an answer, or even mid-way in a recursive loop for self-improvement). The refinement steps themselves can leverage Knowledge Algorithms (KAs) internally – e.g. a “Logic Auditor” KA within the Logical Flow check step ⁴ – but those KAs would be encapsulated inside the step modules or accessible via a shared registry.

In summary, **each refinement step should be a self-contained Python module (or class)** implementing a common interface. The steps are then wired together in a sequence by an orchestrator. This modular design aligns with the system's goals of traceability and rigorous QA, since you can log and trace what each step contributes. It also parallels how the main reasoning layers are modular – e.g. each layer or persona action is a discrete function – enabling the system to be both robust and easily upgradable ¹.

17-Axis System – Modularizing Each Axis with Sub-Workflows and APIs

The **17-axis system** represents knowledge split across 17 distinct dimensions (temporal, spatial, semantic, procedural, analytical, normative, risk, performance, expertise, ethical, regulatory, compliance, simulation, AI knowledge, cultural, innovation, strategic, and learning axes, per the architecture description). To

manage this complexity, it's wise to design the system so that **each axis is handled by dedicated modules or sub-workflows**, which are then integrated via the central knowledge graph and orchestrator. In other words, just as the refinement workflow was broken into modules, the axes can be treated as semi-independent components within the larger system.

Here's how you might approach modularizing each axis:

- **Dedicated Data Pipelines per Axis:** Each axis deals with a particular type of data or knowledge, so you can create separate ingestion and update pipelines for each. For example, a **Temporal Axis module** would handle time-series data or historical events; a **Spatial Axis module** would manage geospatial data; a **Risk Axis module** might ingest risk assessments or incident data, and so on. In the UKG architecture, the knowledge ingestion pipelines are a key part of the system – they continuously pull in and integrate data for their respective domains ⁵ ⁶. Each axis likely has its own ontology and data sources, so having isolated pipeline code for each axis helps maintain clarity. These pipelines can run as separate scheduled jobs or microservices, feeding the unified **Universal Knowledge Graph (UKG)**. The documentation describes these ingestion pipelines as the “circulatory system” of the AI, pumping in data from all quarters, cleaning and normalizing it, and merging it into the UKG ⁶. By separating them, if one axis's data source changes or requires new processing logic, you can update that pipeline without affecting the others.
- **Axis-Specific Logic Modules:** Beyond data ingestion, some axes may require custom logic or analysis routines. For instance, the **Temporal axis** could have a sub-module for temporal reasoning (forecasting timelines, ensuring chronological consistency, implementing the “Arrows of Time” concept for causal ordering) ⁵. The **Compliance/Regulatory axes** might come with rule-checking engines that scan the knowledge graph for violations of policies or laws (this corresponds to the “compliance mesh” module in the architecture) ⁵. Each of these can be implemented as a distinct service or library. By doing so, you essentially give each axis a “brain” or toolset for handling its domain. During query processing or simulation, the main system can invoke these specialized modules when deep analysis in that dimension is needed (for example, calling the temporal reasoner to simulate how a situation might evolve over time, or invoking a risk calculator on the Risk axis data).
- **Encapsulating Axes behind APIs:** If you opt for a microservice architecture, each axis's module could be exposed via well-defined API endpoints. For example, a **Spatial Service** might provide an API to answer proximity or location-based questions (“find all relevant nodes within region X”), while a **Risk Service** could expose an endpoint to calculate risk scores for a scenario. The 17-axis architecture is amenable to this – it uses an event-driven, microservice-like runtime where the orchestrator (and a job router) can direct tasks to the appropriate component or service ². Each axis service would subscribe to relevant events or requests. For instance, when the Query Interpretation layer identifies that a question has a temporal aspect, it could trigger a call to the Temporal service to fetch timeline data or perform a chronology check. The orchestrator ensures these calls happen in the correct sequence and that the results are merged into the global context ². This way, axes can run in parallel or independently as needed, yet contribute to the shared understanding.
- **Unified Knowledge Graph Integration:** Whether or not you use separate services, **all axes must feed into and read from the central knowledge graph**. The UKG serves as the “single source of

truth” that links all 17 dimensions ⁷ . Each axis’s module should publish its data into the graph using a common ontology schema, and likewise query the graph to get relevant multi-axis context. For example, the People axis and Organizations axis might both contribute nodes that link to entries in the Compliance axis (policies, incidents) and the Time axis (dates of events). The architecture explicitly mentions that each axis is integrated via a common ontology and cross-axis links in the graph ⁷ . Practically, this means your axis modules might use a shared library or API to the graph database: e.g., the Risk module writes risk events and links them to relevant entities (temporal nodes for incident date, procedural nodes for process involved, etc.). Entity resolution across axes is crucial – the system needs to know that “Customer X” in the Customer axis is the same as “Client X” mentioned in a Compliance report, for instance. The design includes an entity resolution layer to handle cross-axis linking so that the modules don’t create siloed data ⁸ ⁹ .

- **Axis Sub-Workflows:** Each axis might also have its own internal workflow or state machine, especially during query answering. For example, consider a complex query: *“Should we launch Product Y next quarter, given the market trends and regulatory environment?”* The system will parse this and determine relevant axes: Strategic, Performance, Sector (market trends fall under performance/sector), Temporal (next quarter), Regulatory, etc. The orchestrator could spawn sub-tasks or sub-workflows for each of these axes. One sub-workflow might gather market trend data (Performance axis) and analyze it, another might check upcoming regulatory changes (Regulatory axis), a third might project timelines (Temporal axis). Each sub-workflow could be a sequence of steps or calls within that axis’s domain (e.g., the Regulatory axis workflow might involve: retrieve laws -> find relevant upcoming regulations -> assess compliance impact). Once these sub-tasks complete, their results are merged by the higher-level reasoning layer (often via the persona collaboration layer). This parallel or asynchronous handling is something the system was designed to support – layers 3–5 in the reasoning stack allow concurrent agent (or axis) processing ¹⁰ ² . In implementation terms, you might use asynchronous calls or an event bus to run multiple axis modules at once and then join their results. The **StateGraph** pattern described for UKG would let you model this: each axis module could be a node or a subgraph in the overall state graph, with edges defining how results feed into the next stage.
- **Central Orchestration & Coordination:** Ultimately, a top-level orchestrator or master workflow coordinates all these axis-specific modules. In the 17-axis system, the “Execution and Orchestration” layer handles this role ¹¹ ² . It knows the overall process: parse query into axis coordinates, invoke retrieval across axes, run persona reasoning, then refinement, etc. The YAML configuration snippet in the design showed that the master workflow has references to an *axis schema config*, the simulation stack, and the refinement engine as separate pieces ¹⁰ . This means the orchestrator is aware of the 17-axis schema and will use it in Step 1 (query interpretation to map the question into those axes), and then use the axis information to drive subsequent steps. By having each axis as a module, the orchestrator can dynamically route tasks. For example, *if* a query touches only 5 out of 17 axes, it might only call those five axis services to fetch context, rather than all seventeen. This dynamic routing makes the system efficient and scalable.

Summary: Each of the 17 axes can be thought of as a **modular subsystem** within the larger UKG architecture. In practice you would likely implement them as a combination of: (a) **data/knowledge base modules** (with their own ontology and ingestion pipeline), and (b) **logic/analysis modules** (services or libraries for axis-specific reasoning). All modules connect through the UKG and are supervised by the orchestrator. This yields a cohesive system where axes are distinct (you can update the Temporal reasoning

module without affecting the others) yet tightly integrated. The architecture explicitly highlights modules like a Temporal “Arrows of Time” engine and a Compliance mesh – these correspond to individual axis-focused components ⁵ . By exposing clear APIs or interfaces for each axis module, the rest of the system (personas, KAs, or orchestrator) can invoke them when needed, achieving a clean **multi-axis workflow**. The result is a holistic AI that can “consider a problem from seventeen dimensions” by leveraging specialized sub-systems for each axis and then fusing their insights ⁷ ¹² .

Overall, adopting a **modular design for both the 12-step refinement loop and the 17-axis knowledge system** is crucial for building a complex AI like UKG. It improves maintainability, allows parallel development (different teams can work on different steps or axes), and matches the layered, multi-agent architecture of the system. As long as you define stable interfaces (context objects, graph schemas, event formats) between these modules, you can treat each refinement step or axis component as a plug-in to the core engine. This is exactly how the UKG architecture was envisioned – a collection of focused modules orchestrated in unison to produce a highly confident, cross-domain answer ² ¹¹ .

Sources:

- UKG system design documents and architecture descriptions for 12-step refinement and 17-axis integration ¹³ ¹ ⁷ ⁵ ⁶ ² .

¹ ⁴ ¹³ Universal Knowledge Graph (UKG) MVP – System Documentation.pdf

file:///file-XPhwXoqRAPKvH52RTsRvsT

² ⁵ ⁶ ⁷ ⁸ ⁹ ¹¹ ¹² The_17_Axis_System__Comprehensive_Technical_Architecture.pdf

file:///file-DnK4G9ZbtMW6H3DKJtD84X

³ ¹⁰ ukg_yaml_v2_all_documents.pdf

file:///file-CbAXPe8UdC8kCxFi1x2uJG